The Complete Python Bootcamp: From Zero to Hero in Python (Jose Portilla)

SECTION 1: Course Introduction

Key Takeaways

- Python is beginner-friendly and versatile.
- Used for automation, web development, data science, AI, scripting, testing, and DevOps.
- Python emphasizes readability and simplicity.

SECTION 2: Python Setup

Install Python

python --version

Run Python File

python app.py

Python Interpreter

print("Hello World")

Remember

- `.py` = Python file
- Interpreter executes code line-by-line

SECTION 3: Python Objects & Data Structures

Numbers

Integer

a = 10

Float

b = 10.5

Arithmetic Operators

+
-
*
/
//
%
**

Example:

```
10 // 3   # 3
10 % 3    # 1
2 ** 3    # 8
```

Important

- / → Float division
- / / → Integer division
- % → Remainder

Variables

name = "Kashish"
age = 25

Rules

- Cannot start with number
- Case sensitive
- Use meaningful names

Strings

Create String

name = "Python"

Indexing

name[0]

Slicing

name[0:3]

Common Methods

name.upper()
name.lower()
name.split()

Formatting

Old:

"My name is {}".format(name)

Preferred:

f"My name is {name}"

Remember

Strings are immutable.

Lists

my_list = [1,2,3]

Important Methods

append()
pop()
sort()
reverse()

Nested Lists

matrix = [[1,2],[3,4]]

Access:

matrix[0][1]

Remember

Lists are mutable.

Dictionaries

```
student = {
   "name":"Kashish",
   "age":25
}
```

Access:

student["name"]

Methods:

keys()
values()
items()

Remember

Stores data as key-value pairs.

Tuples

t = (1,2,3)

Methods:

count()
index()

Remember

Immutable version of list.

Sets

s = {1,2,2,3}

Result:

{1,2,3}

Remember

Automatically removes duplicates.

Booleans

True
False

Example:

5 > 2

Result:

True

SECTION 4: Comparison Operators

==
!=
>
<
>=
<=

Example:

5 == 5

Remember

= assigns value
== compares values

SECTION 5: Statements & Control Flow

If-Else

```
if age >= 18:
    print("Adult")
else:
```

```
    print("Minor")
```

Multiple Conditions

```
if marks > 90:
    print("A")
elif marks > 70:
    print("B")
else:
    print("C")
```

Logical Operators

and
or
not

Example:

age > 18 and age < 60

SECTION 6: Loops

For Loop

```
for x in range(5):
    print(x)
```

Common Uses

- Lists
- Strings
- Dictionaries

While Loop

```
x = 0

while x < 5:
    x += 1
```

Loop Keywords

break

Stops loop.

continue

Skips iteration.

pass

Placeholder.

SECTION 7: Useful Operators

Range

```
range(5)
```

Output:

0 1 2 3 4

Enumerate

```
for i,v in enumerate(['a','b']):
    print(i,v)
```

Output:

0 a
1 b

Zip

list(zip([1,2],[3,4]))

Output:

[(1,3),(2,4)]

---

in Operator

"a" in "apple"

Output:

True

List Comprehension ⭐

[x for x in range(5)]

With Condition:

[x for x in range(10) if x%2==0]

Most used Python feature.

SECTION 8: Methods & Functions

Function

```
def greet(name):
    return f"Hello {name}"
```

Return vs Print

return

returns value.

print

only displays value.

---

*args

```
def add(*args):
    return sum(args)
```

Accepts unlimited positional arguments.

**kwargs

```
def info(**kwargs):
    print(kwargs)
```

Accepts unlimited keyword arguments.

SECTION 9: Lambda Expressions, Map & Filter

Lambda

lambda x:x*x

Small anonymous function.

Map

list(map(lambda x:x*2,[1,2,3]))

Transforms every item.

Filter

list(filter(lambda x:x%2==0,[1,2,3,4]))

Filters matching items.

SECTION 10: Scope

LEGB Rule

1. Local
2. Enclosing
3. Global
4. Built-In

Example:

```
x = 10

def test():
    print(x)
```

Python searches variables in LEGB order.

SECTION 11: Object Oriented Programming ⭐⭐⭐

Class

Blueprint.

```
class Dog:
    pass
```

Constructor

```
def __init__(self,name):
    self.name = name
```

Runs automatically during object creation.

Object

```
dog1 = Dog("Tommy")
```

Methods

```
def bark(self):
```

```
    print("Woof")
```

Object behavior.

Inheritance

```
class Animal:
    pass

class Dog(Animal):
    pass
```

Code reuse.

Polymorphism

Same method name.

Different implementation.

Special Methods

__init__
__str__
__len__

Customize object behavior.

SECTION 12: Modules & Packages

Module

```
import mymodule
```

Single Python file.

Package

```
package/
   __init__.py
   module1.py
```

Collection of modules.

SECTION 13: Errors & Exception Handling

```
try:
   risky_code()

except:
   handle_error()

finally:
   cleanup()
```

Remember

- try → execute
- except → handle error
- finally → always run

SECTION 14: File Handling

Read

```
with open("file.txt","r") as f:
    print(f.read())
```

Write

```
with open("file.txt","w") as f:
    f.write("Hello")
```

Remember

Always use:

with open(...)

SECTION 15: Advanced Python Modules

Common modules introduced:

Collections

```
from collections import Counter
```

Count occurrences.

Datetime

```
import datetime
```

Work with dates and time.

Math

```
import math
```

Math functions.

Random

import random

Generate random values.

SECTION 16: Decorators ⭐⭐

Modify function behavior.

```
@decorator
def hello():
    pass
```

Used for:

- Logging
- Authentication
- Monitoring

SECTION 17: Generators ⭐⭐

Use:

yield

instead of:

return

Example:

```
def count():
    for i in range(5):
        yield i
```

Benefits

- Memory efficient
- Faster for large data

SECTION 18: Regular Expressions (Regex)

```
import re
```

Search patterns.

```
re.search(r"\d+","abc123")
```

Patterns:

```
\d  digit
\w  word
\s  space
+   one or more
*   zero or more
```

SECTION 19: Working with PDFs, CSVs & Files

CSV

```
import csv
```

Read/write CSV files.

PDF

import PyPDF2

Extract PDF content.

Use Cases

- Reports
- Data processing
- Automation

SECTION 20: Emails

import smtplib

Send emails using Python.

Common use:

- Notifications
- Alerts
- Automation workflows

SECTION 21: Web Scraping

Libraries:

requests
beautifulsoup4

Example Workflow:

requests.get()

↓

BeautifulSoup()

↓

Extract Data

Uses

- News scraping
- Product data
- Reports

SECTION 22: Working with Images

Library:

Pillow

Example:

from PIL import Image

Tasks:

- Resize
- Crop
- Modify images
